Successors to Transformer

Jiaqi Xia

University of Science and Technology of China

2025.5.13

1 Background

2 Mamba

3 Retentive Network

4 References

1 Background

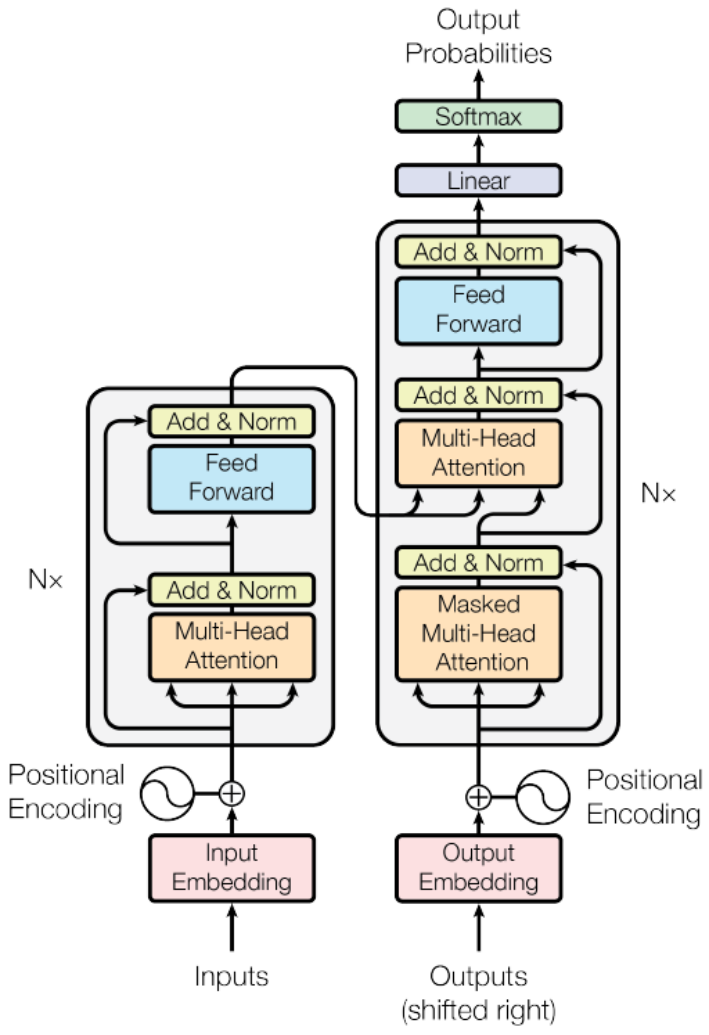
2 Mamba

3 Retentive Network

4 References

Background

Transformer has become the defacto architecture for large language models.



Background

The efficacy of self-attention is attributed to its ability to route information densely within a context window, allowing it to model complex data.

However, this property brings fundamental drawbacks: an inability to model anything outside of a finite window, and quadratic scaling with respect to the window length.

Mamba

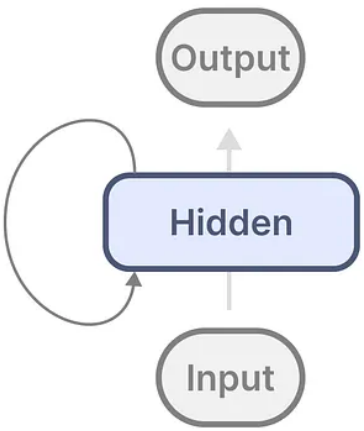
Recurrent neural network

The recurrent neural network (RNN):

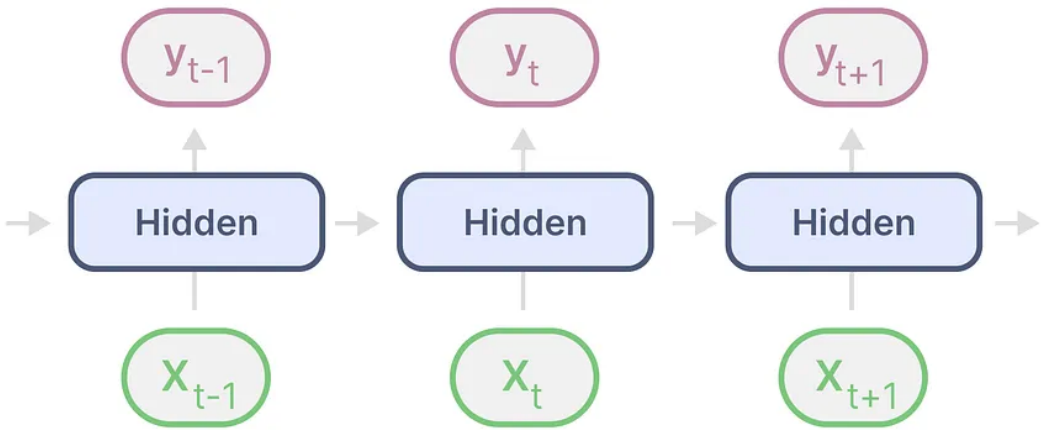
$$h_t = f(Wh_{t-1} + Ux_t)$$

$$y_t = g(Vh_t)$$

Here x_t is the input, y_t is the output, h_t is the hidden state, f, g are functions.



RNN



RNN
(Unfolded)

1 Background

2 Mamba

3 Retentive Network

4 References

Mamba

State space models

The continuous state space model:

$$h'(t) = Ah(t) + Bx(t)$$
$$y(t) = Ch(t)$$

To facilitate sequence modelling, a possible discretization follows:

$$\begin{aligned} h(t+\triangle) &\approx \triangle h'(t) + h(t) \\ &= \triangle Ah(t) + \triangle Bx(t) + h(t) \\ &= (I + \triangle A)h(t) + \triangle Bx(t) \\ &= \overline{A}h(t) + \overline{B}x(t) \end{aligned}$$

where $\triangle$ is a parameter.

Mamba

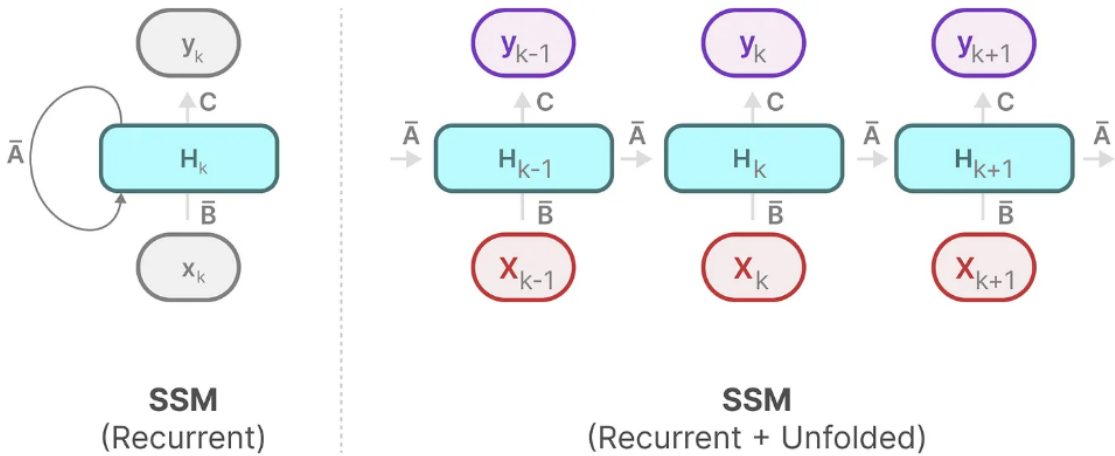
State space models

General discretized form:

$$h_t = \overline{A}h_{t-1} + \overline{B}x_t$$

$$y_t = Ch_t$$

where $\overline{A} = f_A(\triangle, A)$, $\overline{B} = f_B(\triangle, B)$.



Mamba

State space models

Assume $x_{-1} = 0$, then

$$h_0 = \overline{B}x_0$$
$$h_1 = \overline{A}\overline{B}x_0 + \overline{B}x_1$$
$$h_2 = \overline{A}(\overline{A}\overline{B}x_0 + \overline{B}x_1) + \overline{B}x_2$$
$$h_3 = \overline{A}(\overline{A}(\overline{A}\overline{B}x_0 + \overline{B}x_1) + \overline{B}x_2) + \overline{B}x_3$$

Thus

$$y_3 = C\overline{A}^3\overline{B}x_0 + C\overline{A}^2\overline{B}x_1 + C\overline{A}\overline{B}x_2 + C\overline{B}x_3$$

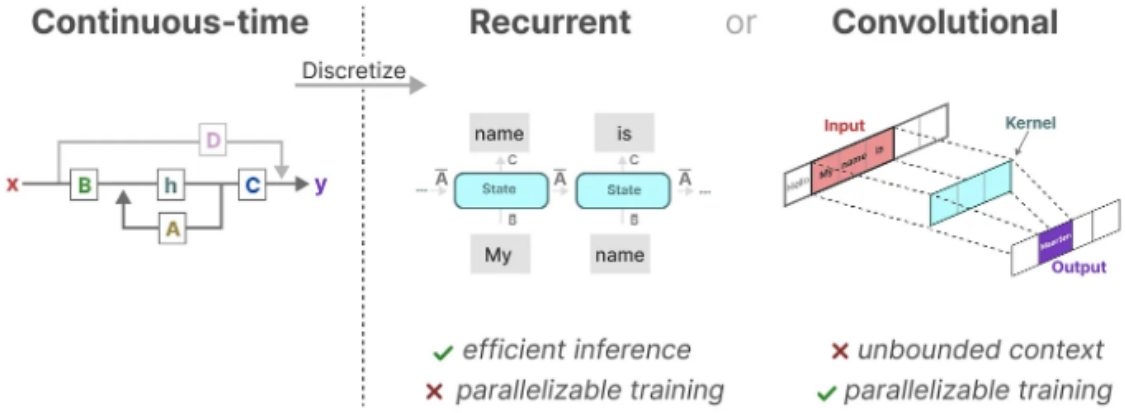
Moreover, we get the convolution representation:

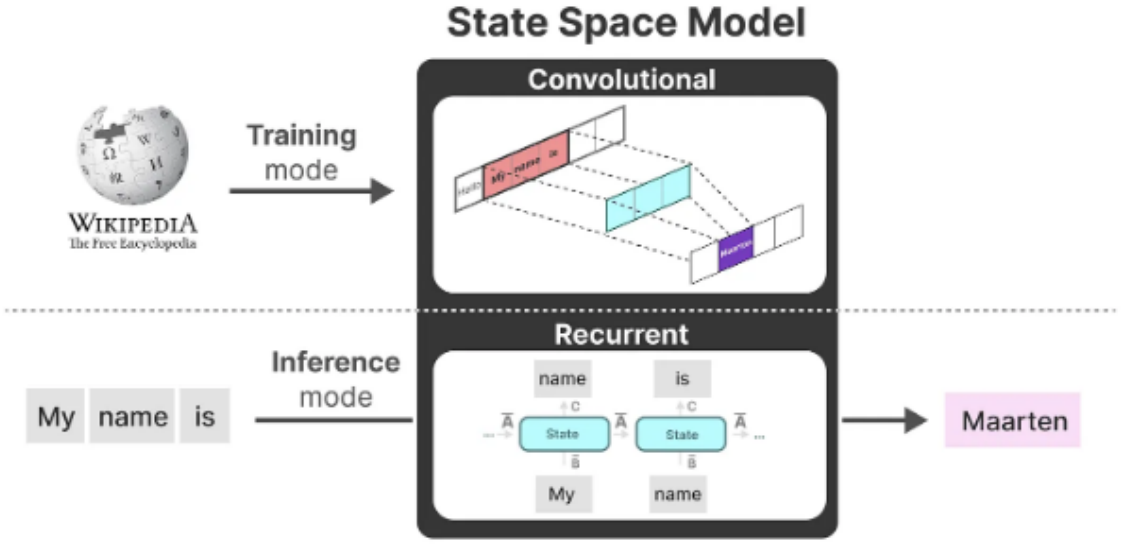
$$y = x * \overline{K}$$

where $\overline{K} = (C\overline{B} \quad C\overline{A}\overline{B} \quad \cdots \quad C\overline{A}^k\overline{B})$

Mamba

State space models

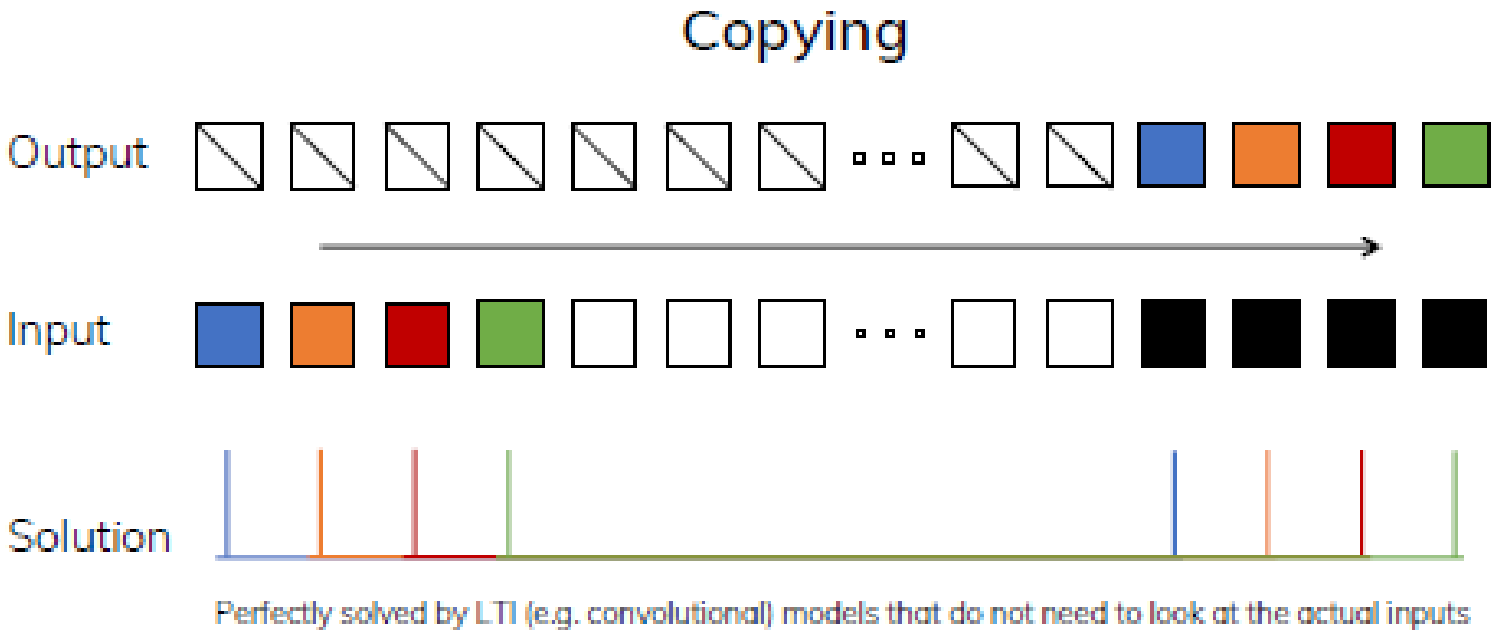




Mamba

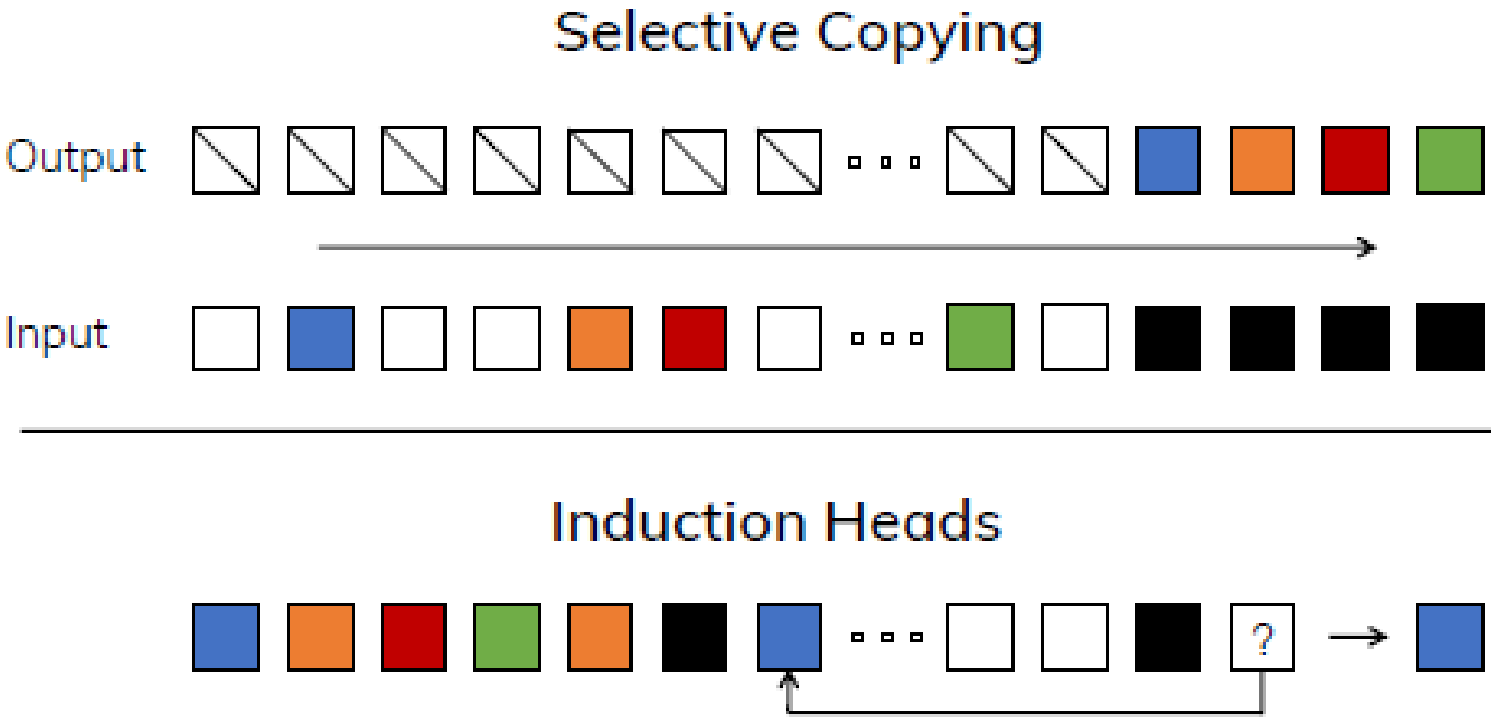
Improving SSMs with selection

The parameters in the state space model is time invariant, rendering it fail to deal with content-aware synthetic tasks.



Mamba

Improving SSMs with selection



Mamba

Improving SSMs with selection

B: batch size, L: sequence length, D: dimension of input, N: dimension of h.

Algorithm 1 SSM (S4)

Input: $x : (\mathsf{B}, \mathsf{L}, \mathsf{D})$
Output: $y : (\mathsf{B}, \mathsf{L}, \mathsf{D})$
1: $A : (\mathsf{D}, \mathsf{N}) \leftarrow$ Parameter
▷ Represents structured $N \times N$ matrix
2: $B : (\mathsf{D}, \mathsf{N}) \leftarrow$ Parameter
3: $C : (\mathsf{D}, \mathsf{N}) \leftarrow$ Parameter
4: $\Delta : (\mathsf{D}) \leftarrow \tau_\Delta(\text{Parameter})$
5: $\overline{A}, \overline{B} : (\mathsf{D}, \mathsf{N}) \leftarrow \text{discretize}(\Delta, A, B)$
6: $y \leftarrow \text{SSM}(\overline{A}, \overline{B}, C)(x)$
▷ Time-invariant: recurrence or convolution
7: **return** y

Algorithm 2 SSM + Selection (S6)

Input: $x : (\mathsf{B}, \mathsf{L}, \mathsf{D})$
Output: $y : (\mathsf{B}, \mathsf{L}, \mathsf{D})$
1: $A : (\mathsf{D}, \mathsf{N}) \leftarrow$ Parameter
▷ Represents structured $N \times N$ matrix
2: $B : (\mathsf{B}, \mathsf{L}, \mathsf{N}) \leftarrow s_B(x)$
3: $C : (\mathsf{B}, \mathsf{L}, \mathsf{N}) \leftarrow s_C(x)$
4: $\Delta : (\mathsf{B}, \mathsf{L}, \mathsf{D}) \leftarrow \tau_\Delta(\text{Parameter} + s_\Delta(x))$
5: $\overline{A}, \overline{B} : (\mathsf{B}, \mathsf{L}, \mathsf{D}, \mathsf{N}) \leftarrow \text{discretize}(\Delta, A, B)$
6: $y \leftarrow \text{SSM}(\overline{A}, \overline{B}, C)(x)$
▷ Time-varying: recurrence (*scan*) only
7: **return** y

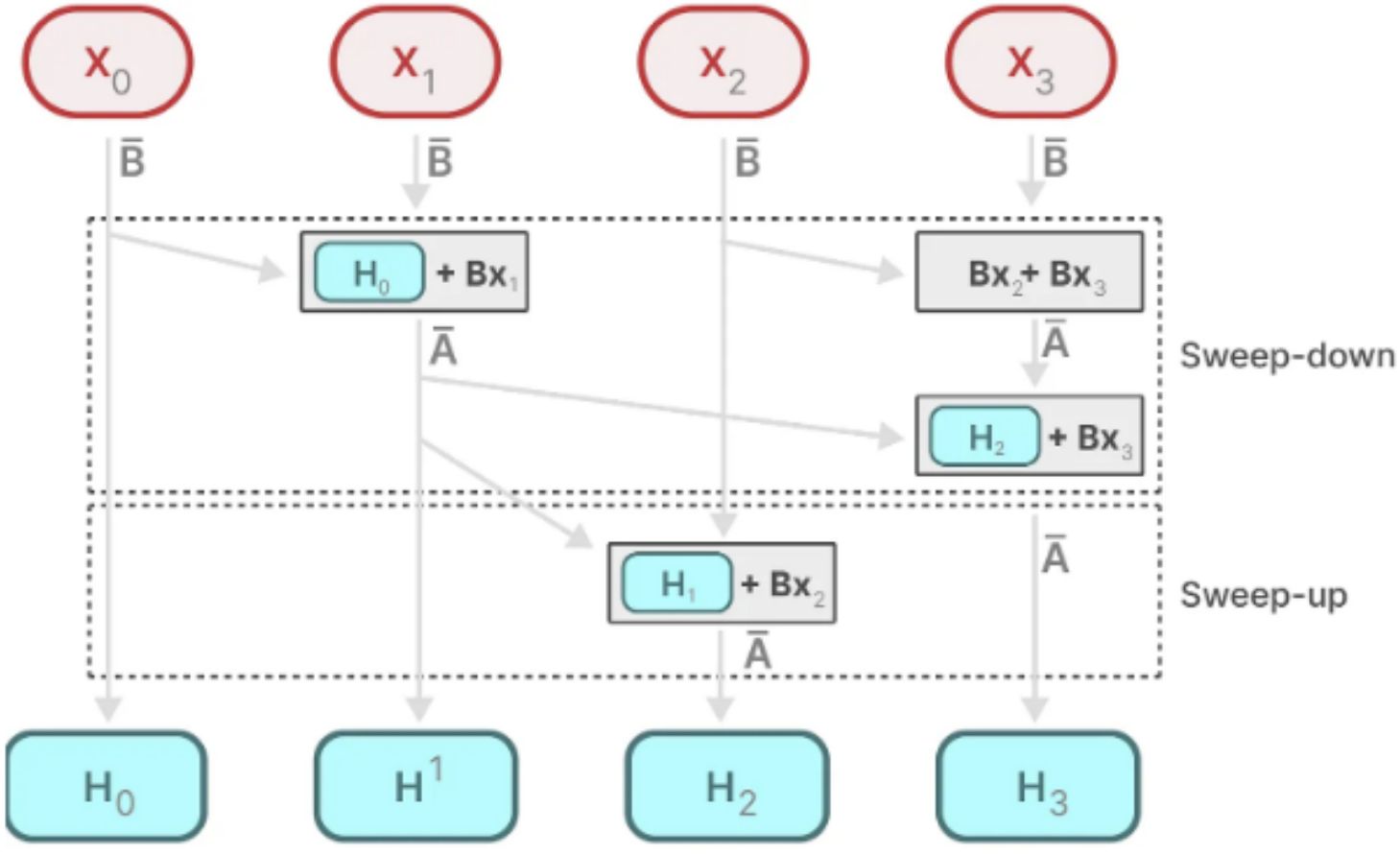
Here $S_B(x) = \text{Linear}_N(x)$, $S_C(x) = \text{Linear}_N(x)$, $S_\triangle(x) = \text{Broadcast}_D(\text{Linear}_1(x))$ and $\tau_\triangle = \text{softplus}$, where $\text{Linear}_d(x)$ is a parameterized projection to dimension d.

* Parallel computation is not available after selection.

Mamba

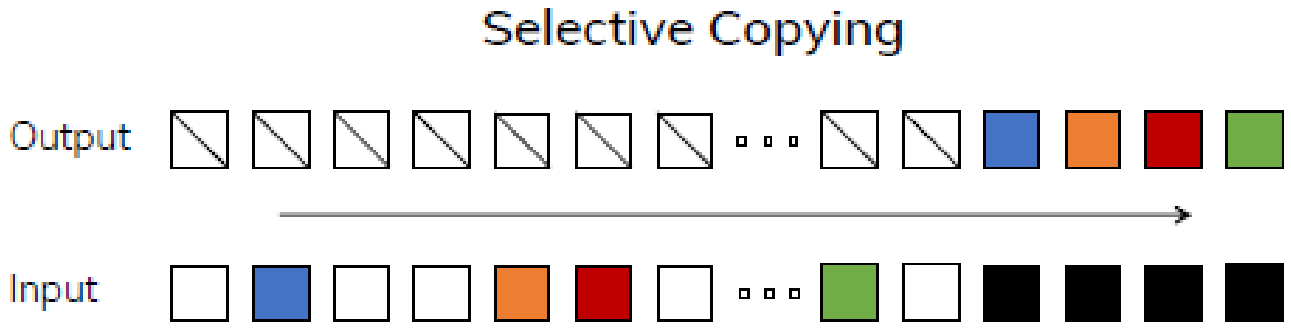
Parallel scan

Parallel scan:



Mamba

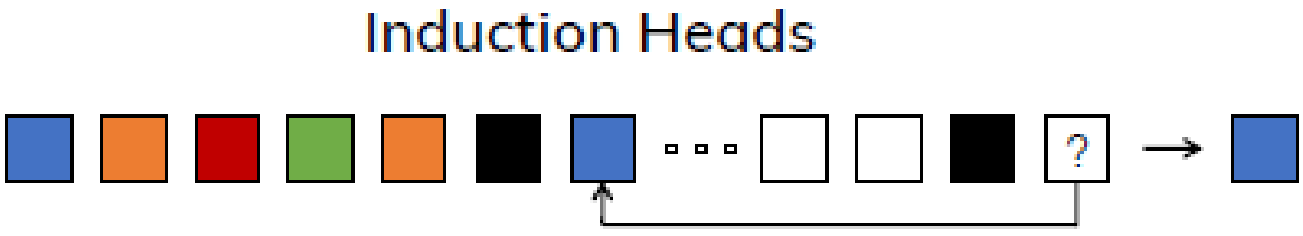
Experiments

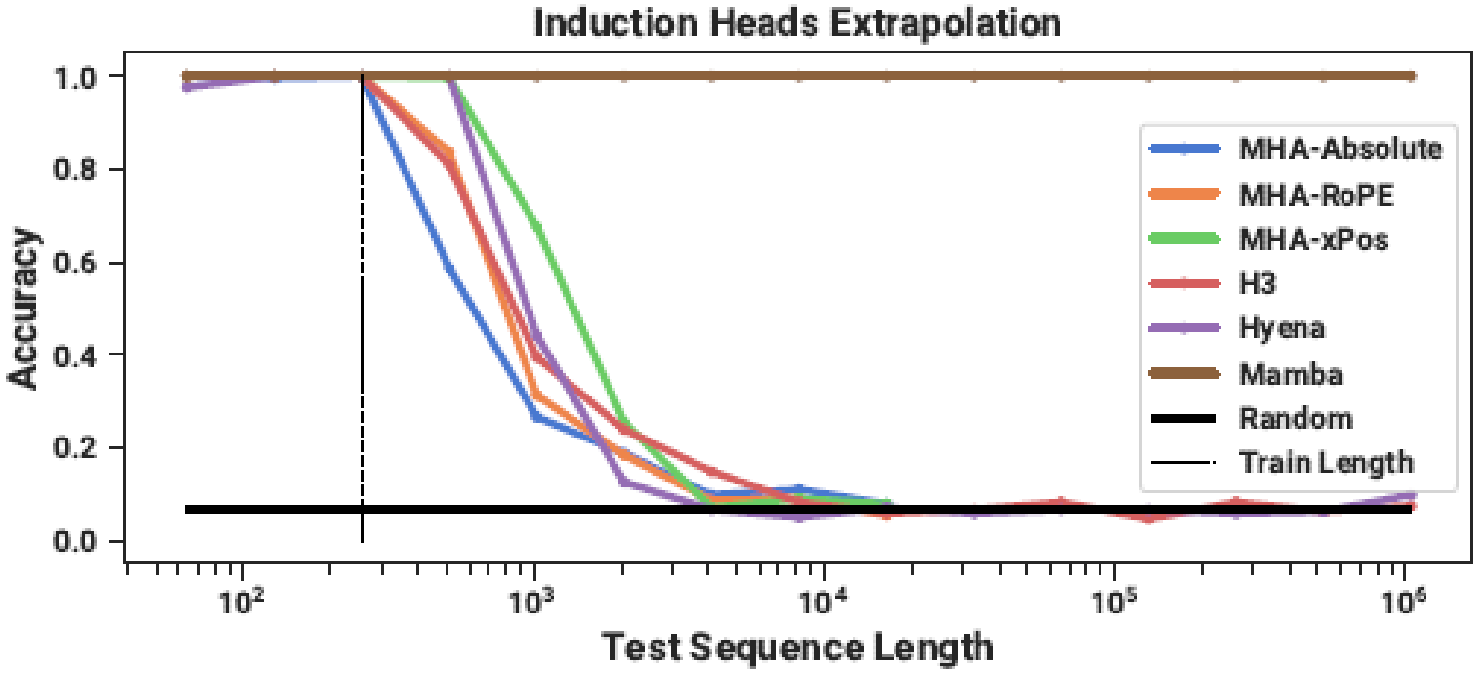


Model	Arch.	Layer	Acc.
S4	No gate	S4	18.3
-	No gate	S6	**97.0**
H3	H3	S4	57.0
Hyena	H3	Hyena	30.1
-	H3	S6	**99.7**
-	Mamba	S4	56.4
-	Mamba	Hyena	28.4
Mamba	Mamba	S6	**99.8**

Mamba

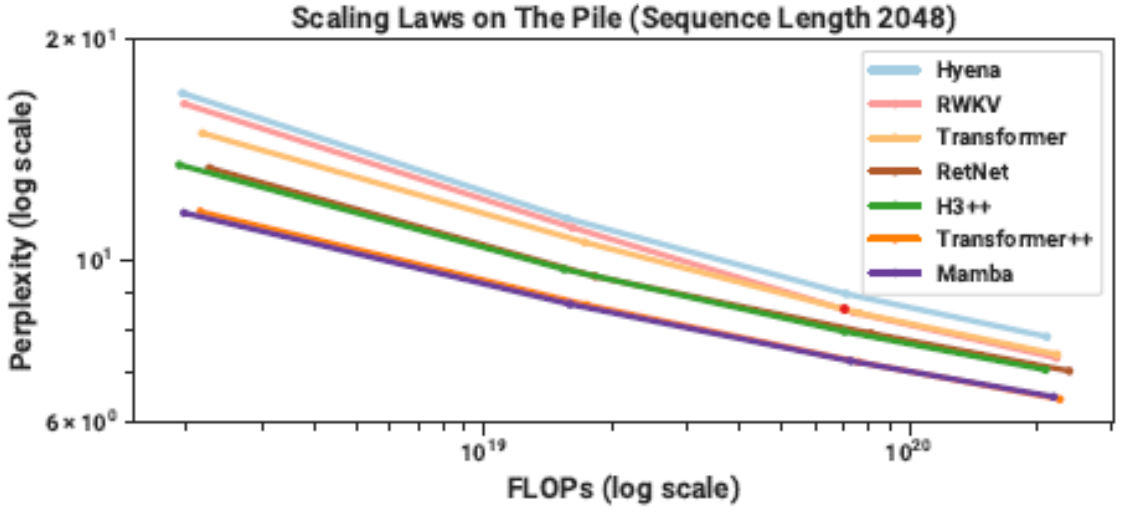
Experiments

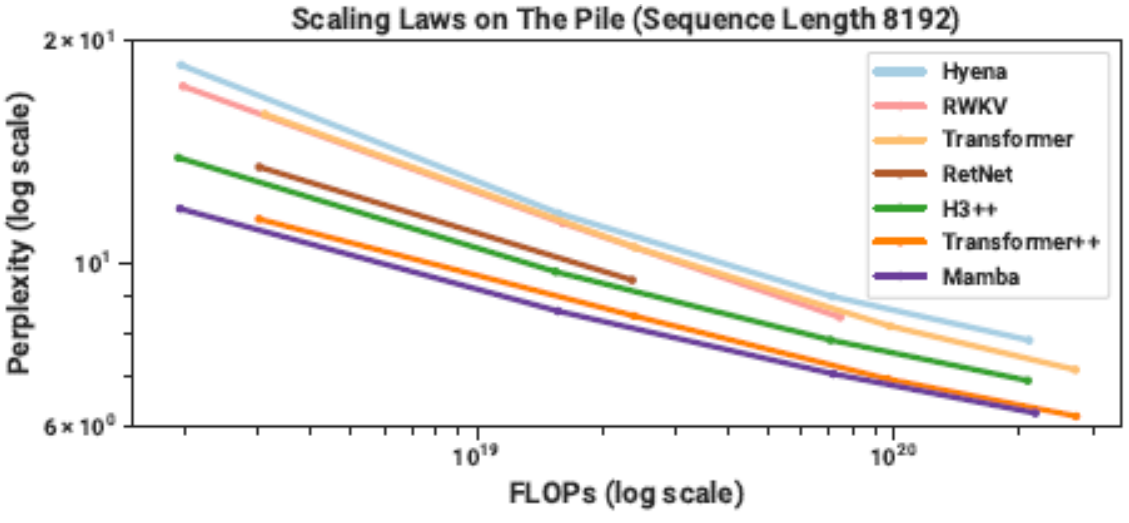




Mamba

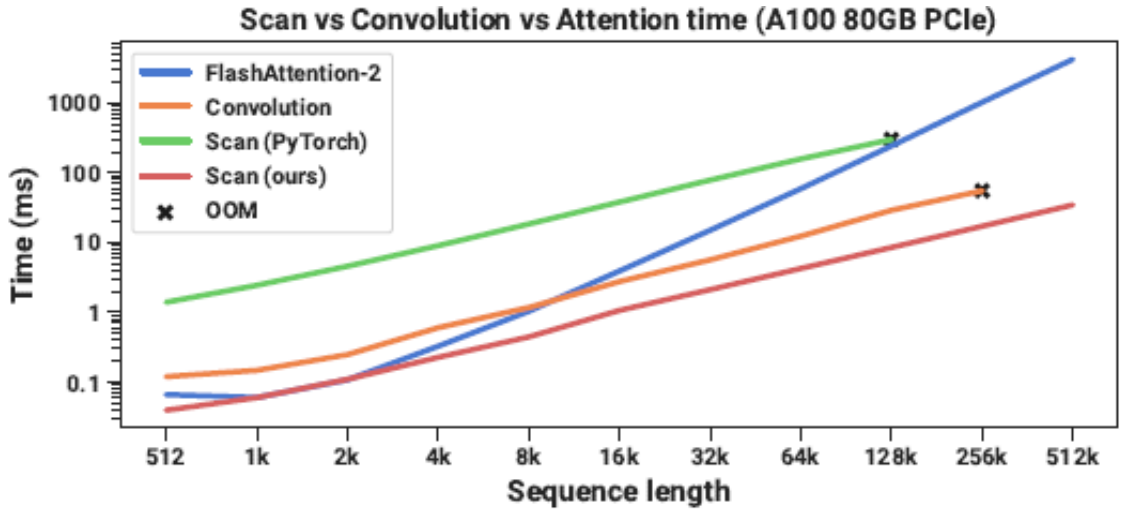
Experiments

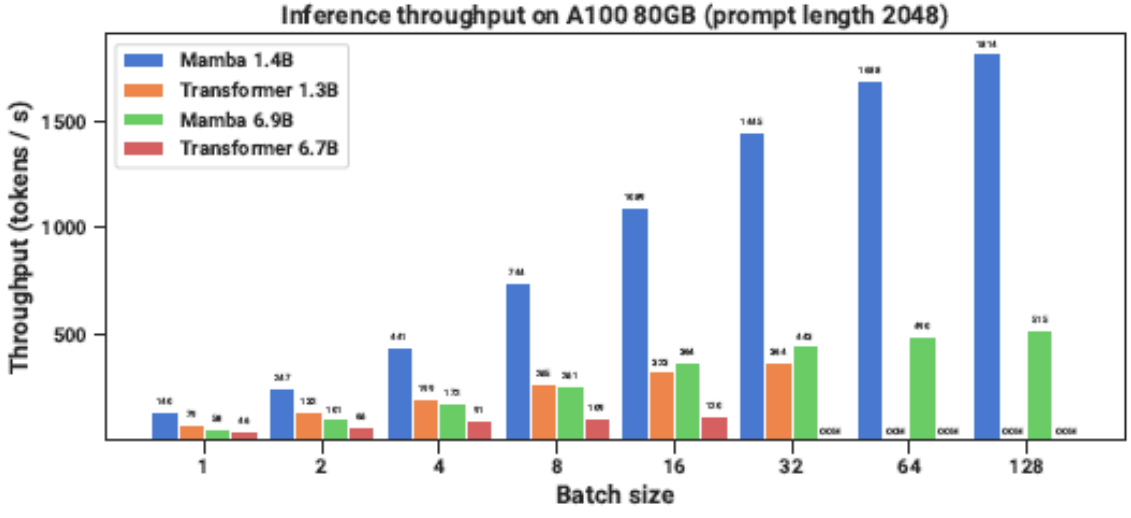




Mamba

Experiments





1 Background

2 Mamba

3 Retentive Network

4 References

Retentive Network

The retention mechanism

Input: $X = [x_1, \cdots, x_{|x|}] \in \mathbb{R}^{|x| \times d_{\text{model}}}$

Projection: $v_n = X_n \cdot w_V$, $w_V \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$

Modeling: $v_n \mapsto o_n$ (through states s_n)

$$s_n = A s_{n-1} + K_n^T v_n, \quad A \in \mathbb{R}^{d \times d}, K_n \in \mathbb{R}^{1 \times d},$$

$$o_n = Q_n s_n = \sum_{m=1}^{n} Q_n A^{n-m} K_m^T v_m, \quad Q_n \in \mathbb{R}^{1 \times d}$$

Write $Q = XW_Q$, $K = XW_K$ where $W_Q, W_K \in \mathbb{R}^{d \times d}$ are learnable matrices.

Retentive Network

The retention mechanism

If we diagonalize A: $A = \Lambda(\gamma e^{i\theta})\Lambda^{-1}$, $\gamma, \theta \in \mathbb{R}^d$

Then $A^{n-m} = A = \Lambda(\gamma e^{i\theta})^{n-m}\Lambda^{-1}$

Absorbing Λ into W_Q and W_K, then

$$
\begin{aligned}
o_n &= \sum_{m=1}^{n} Q_n(\gamma e^{i\theta})^{n-m} K_m^T v_m \\
&= \sum_{m=1}^{n} (Q_n(\gamma e^{i\theta})^n)(K_m(\gamma e^{i\theta})^{-m})^T v_m \\
&= \sum_{m=1}^{n} \gamma^{n-m}(Q_n e^{in\theta})(K_m e^{im\theta})^\top v_m
\end{aligned}
$$

Retentive Network

Representation

Let $\Theta_n = e^{in\theta}, D = \begin{cases} \gamma^{n-m}, & n \geq m \\ 0, & n < m \end{cases}$

The **parallel representation** of retention:

$$Q = (XW_Q) \odot \Theta,\ K = (XW_K) \odot \overline{\Theta},\ V = XW_V$$

$$\text{Retention}(X) = (QK^T \odot D)V$$

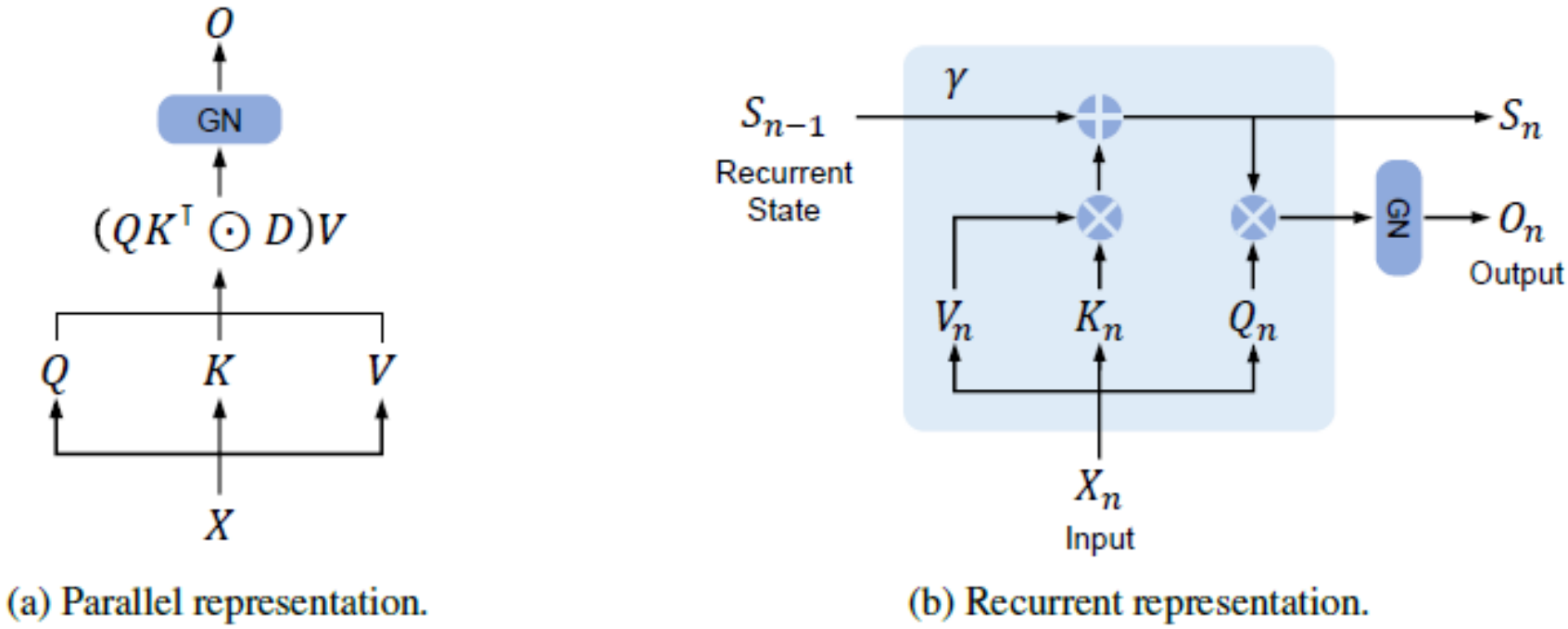
The **recurrent representation** of retention:

$$S_n = \gamma S_{n-1} + K_n^T V_n,$$

$$\text{Retention}(X_n) = Q_n S_n,$$

Retentive Network

Representation



(a) Parallel representation.

(b) Recurrent representation.

Figure: Dual form of RetNet, ”GN” is short for GroupNorm

The parallel representation will be used in training, while the recurrent representation will be used in inference.

Retentive Network

Gated multi-scale retention

The retentive network use $h = d_{\text{model}}/d$ retention heads in each layer.

The heads use different parameter matrices $W_Q, W_K, W_V \in \mathbb{R}^{d \times d}$.

Multi-scale retention assigns different γ for each head.

The layers are defined as:

$$\text{head}_i = \text{Retention}(X, \gamma_i)$$

$$Y = \text{GroupNorm}_h(\text{Concat}(\text{head}_1, \cdots, \text{head}_h))$$

$$\text{MSR}(X) = (\text{swish}(XW_G) \odot Y)W_O$$

where $W_G, W_O \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$ are learnable parameters.

Retentive Network

Overall architecture

An $L-$layer Retentive Network:

$$Y^l = \text{MSR}(\text{LN}(X^l)) + X^l$$

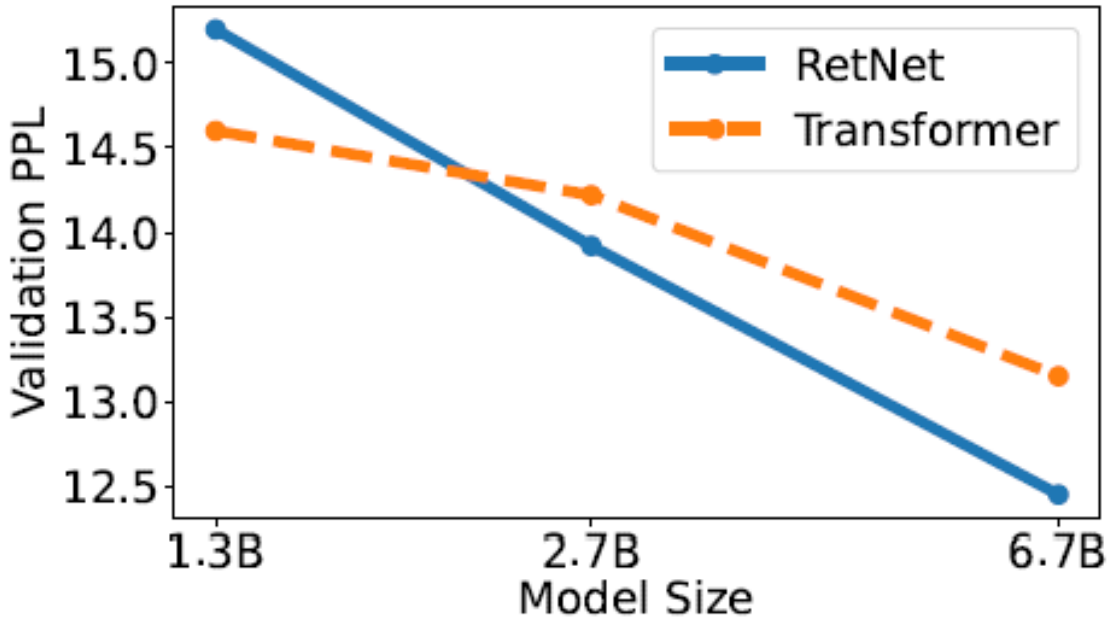
$$X^{l+1} = \text{FFN}(\text{LN}(Y^l)) + Y^l$$

Here $\text{LN}(\cdot)$ is LayerNorm. FFN is feed-forward network computed as $\text{FFN}(X) = \text{gelu}(XW_1)W_2$ where W_1, W_2 are parameter matrices.

Retentive Network

Experiments

Size	Hidden Dim.	#Layers	Batch Size	# Tokens	Learning Rate
1.3B	2048	24	4M	100B	6×10^{-4}
2.7B	2560	32	4M	100B	3×10^{-4}
6.7B	4096	32	4M	100B	3×10^{-4}



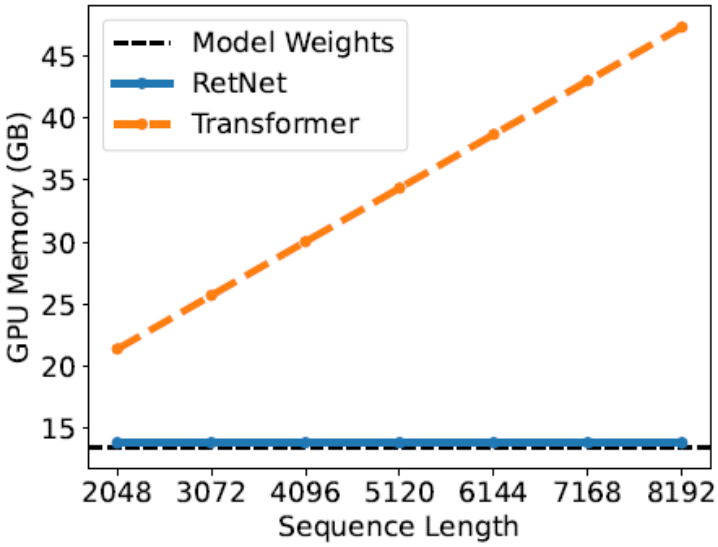
Retentive Network

Experiments

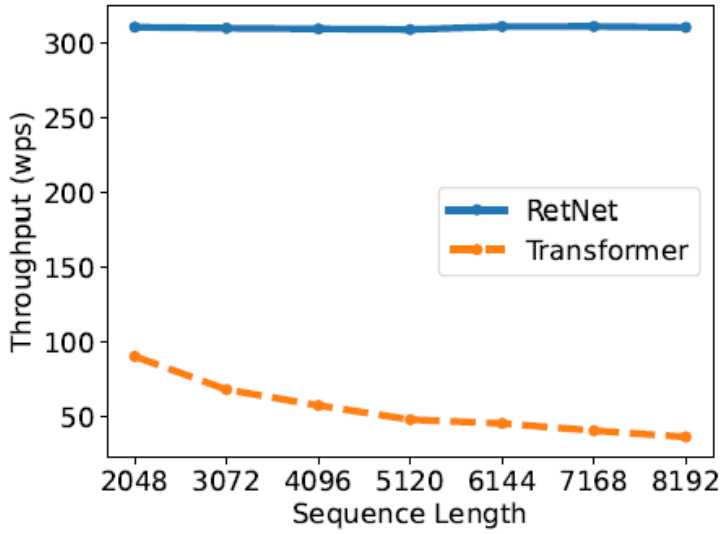
Model Size	Memory (GB) ↓			Throughput (wps) ↑		
	Trm	Trm+FlashAttn	RetNet	Trm	Trm+FlashAttn	RetNet
1.3B	74.8	38.8	34.5	10832.4	63965.2	73344.8
2.7B	69.6	42.1	42.0	5186.0	34990.2	38921.2
6.7B	69.0	51.4	48.0	2754.4	16230.1	17458.6
13B	61.4	46.3	45.9	1208.9	7945.1	8642.2

Retentive Network

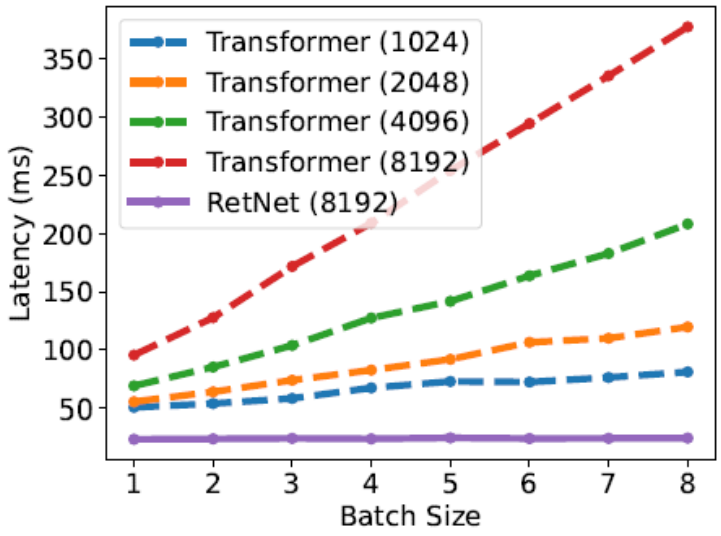
Experiments



(a) GPU memory cost of Transformer and RetNet.



(b) Throughput of Transformer and RetNet.



(c) Inference latency with different batch sizes.

Summary

Architectures	Training Parallelization	Traning Cost	Inference cost
Transformer		$O(N^2)$	$O(N)$
Mamba	✓	$O(N)$	$O(1)$
RetNet	✓	$O(N)$	$O(1)$

1 Background

2 Mamba

3 Retentive Network

4 References

References I

[1] Albert Gu and Tri Dao. *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*. 2024. arXiv: 2312.00752 [cs.LG]. URL: https://arxiv.org/abs/2312.00752.

[2] Yutao Sun et al. *Retentive Network: A Successor to Transformer for Large Language Models*. 2023. arXiv: 2307.08621 [cs.CL]. URL: https://arxiv.org/abs/2307.08621.

[3] Ashish Vaswani et al. *Attention Is All You Need*. 2023. arXiv: 1706.03762 [cs.CL]. URL: https://arxiv.org/abs/1706.03762.